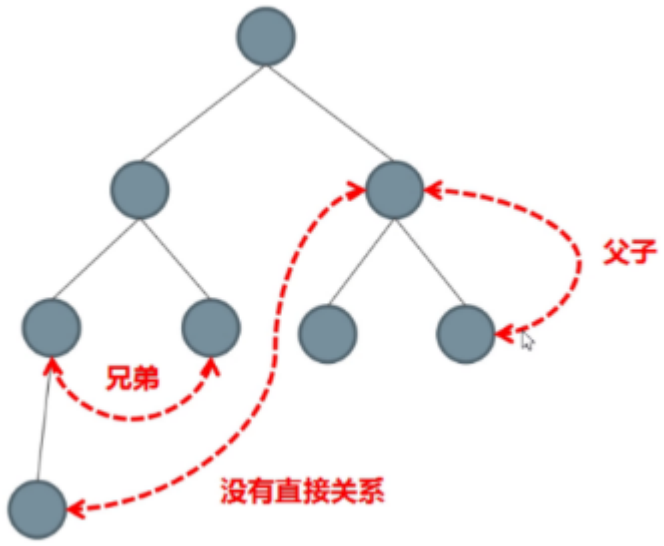
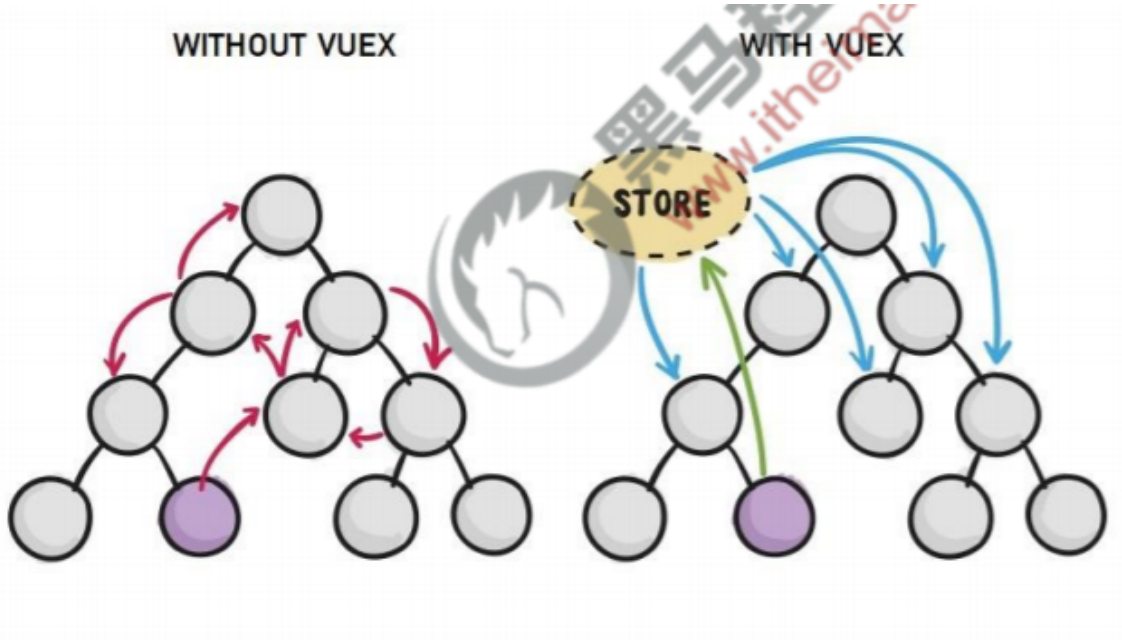
- **vuex介绍**
  - 通信方案



  -

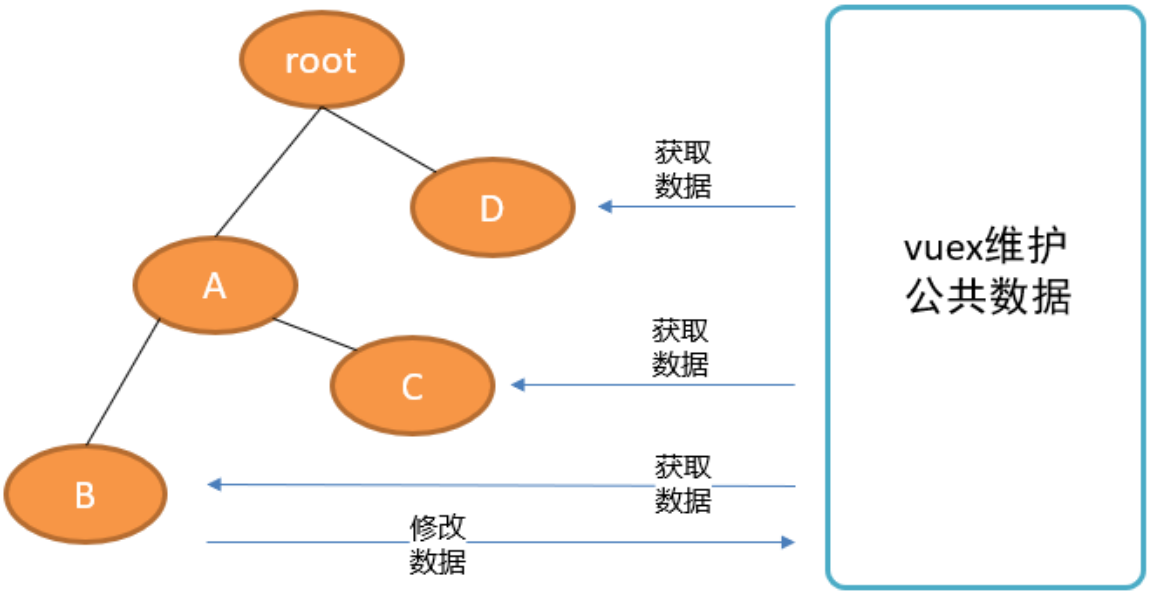
组件关系	数据通信
父子关系	父传子：props ； 子传父：$emit
非父子关系	vuex (一种组件通信方案)

  - vuex是什么
    - Vuex 是一个专为 Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理数据，以相应的规则保证状态以一种可预测的方式发生变化



  - vuex为何学
    - 程序页面多，数据变量多，但又要做到：
      - 不同组件数据保持同步
      - 数据的修改都是可追踪

    - 保持同步、可追踪的含义：
        - 一个户外商店有两名员工，张三和李四。一天的早上，他们分别对帐篷的数量做了一次盘点，发现一共有三个帐篷。张三卖出去俩个，他以为库存里还有一个。李四卖出去一个，他以为库存里还有两个。而事实上是，库存现在已经为零。如果他们再接受客户的预订，就会出现库存不足的情况。
        - 张三和李四因为没有保持库存的数量的同步导致了尴尬，这个就是所谓的`数据保持同步 `
        - 店长需要知道, 谁卖出了多少个帐篷，这个行为我们称之为`数据修改是可追踪的`
    - 图示:

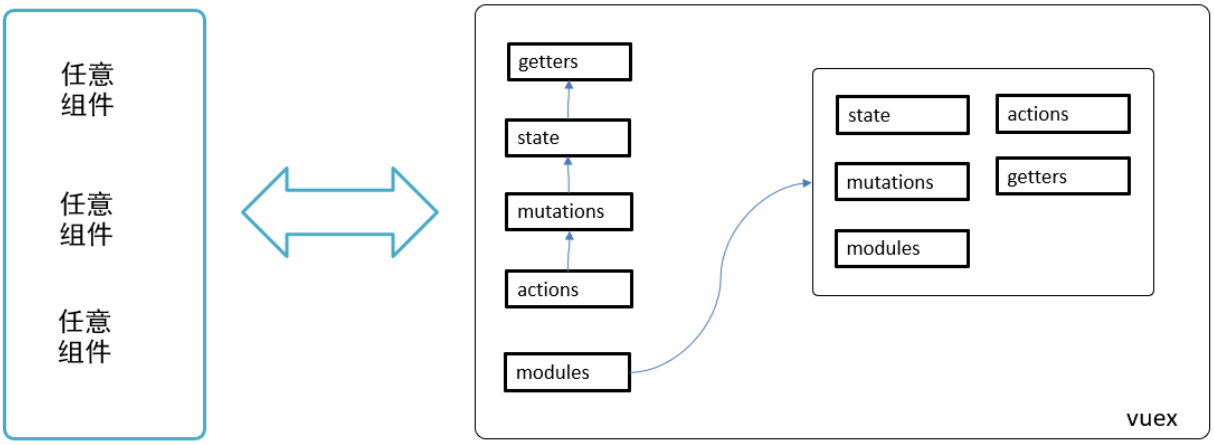


- vuex中存什么
    - 多个组件共享状态，才存储在vuex中
    - 某个组件中的私有数据，依旧存储在data中
    - 例如：
        - 登陆的用户名需要在首页, 个人中心, 结算页面使用, 用户名存在vuex中
        - 文章详情数据, 只有在文章详情页查看, 在自身data中声明
- 小结
    - 什么是vuex
        - vuex是Vue官方推荐的集中式状态管理机制
    - 为何学vuex
        - 数据同步, 集中管理
    - vuex中存什么
        - 多个组件共享的值

- **vuex学习内容**
    - 核心概念
        - 官网地址: https://vuex.vuejs.org/zh/
        - 安装(固定)
        - 配置项(固定)

配置项	含义	注意
state	单一状态树	类似data
mutations	数据管家(同步)	唯一修改state地方
actions	异步请求	要改state需要提交给mutations
getters	vuex计算属性	类似computed
modules	模块拆分	

- 图示关系
  - 单一定义store对象, 里面5个配置项, 在任意组件可以使用



- 小结
  - vuex五个核心概念是?
    - state / mutations / actions / getters / modules

- **vuex-state（数据源）**
  - 定义state
    - 语法:

```
const store = new Vuex.Store({
    state: {
        变量名: 初始值
    }
})
```

    - 具体代码:

```
const store = new Vuex.Store({
    state: {
        count: 100 // 库存
    }
})
```

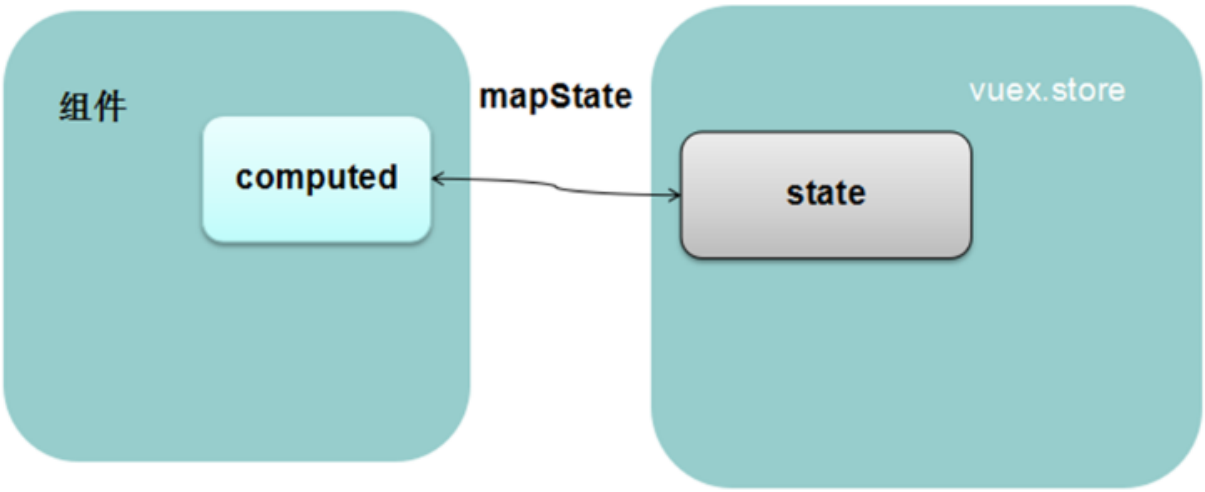
- 使用state的两种方式
  - 直接使用
    - 语法:

```
this.$store.state.变量名
```

  - 映射使用 (推荐)
    - 语法:

```
// 1. 拿到mapState辅助函数
import { mapState } from 'vuex'
export default {
    computed: {
        // 2. 把state里变量映射到计算属性中
        ...mapState(['state里的变量名'])
    }
}
```

    - 整个过程的示意图如下



- 注意
  - state是响应式的, 只要state值变化, 页面上使用的地方会自动更新同步
- 小结
  - state作用?
    - 定义全局状态数据源
  - state如何定义?
    - 在store内, state: {变量名: 初始值}
  - state的值如何用到具体vue组件内?
    - 直接使用 this.$store.state.变量名
    - 映射使用 ...mapState(['state的变量名'])

- **vuex-mutations（同步修改）**
  - 定义mutations
    - 语法:

```
const store  = new Vuex.Store({
    mutations: {
        函数名 (state, 可选值) {
            // 同步修改state值代码
        }
    }
})
```

    - 具体代码

```
const store  = new Vuex.Store({
    state: {
        count: 100 // 库存
    },
    mutations: {
        addCount (state, value) { // 负责增加库存的管家
            state.count += value
        },
        subCount (state, value) { // 负责减少库存的管家
            state.count -= value
        },
        setCount (state, value) { // 负责直接修改库存的管家
            state.count = value;
```

```
        }
    }
})
```

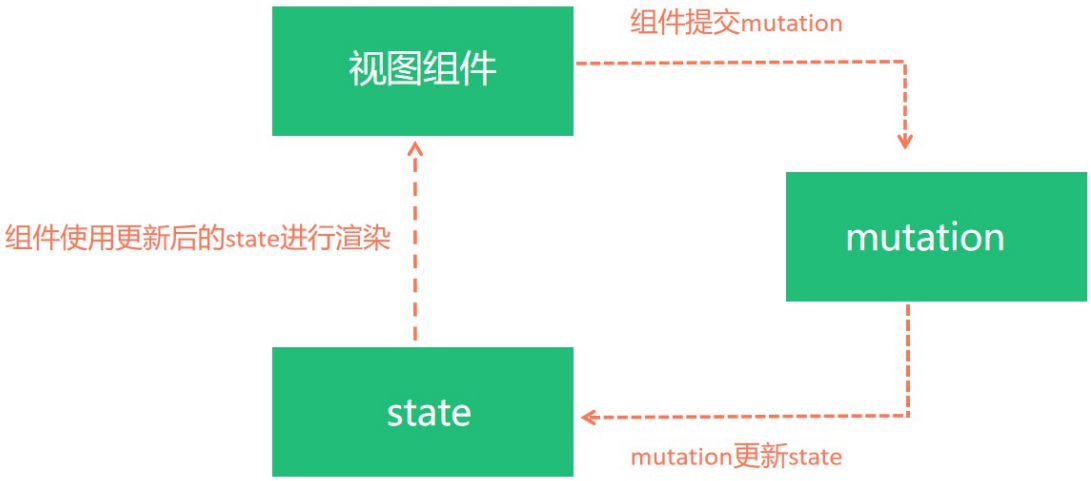
- 使用mutations的两种方式
  - 直接使用
    - 语法:

```
this.$store.commit("mutations里的函数名", 具体值)
```

  - 映射使用
    - 语法:

```
// 1. 拿到mapMutations辅助函数
import { mapMutations } from 'vuex'
export default {
    methods: {
        // 2. 把mutations里方法映射到原地
        ...mapMutations(['mutations里的函数名'])
    }
}
```

- 注意
  - mutations是唯一能修改state的地方, 确保调试工具可以追踪变化
  - mutations函数内, 只能写同步代码, 调试工具可追踪变化过程
    - 因为调试工具要立刻产生一次记录, 所以必须是同步的
  - mutations函数上, 只能接收一个参数值, 如果传多个, 请传一个对象
- 小结
  - mutations里函数作用?
    - 负责修改state里的数据
  - mutations只能写什么样的代码?
    - 同步流程的代码
  - mutations有哪两种使用方式?
    - 直接使用 this.$store.commit()
    - 映射使用 mapMutations把方法映射到组件内直接调用
  - state, mutations, 视图组件, 3个关系是什么?



- **vuex-actions（异步修改）**
  - 定义actions
    - 语法:

```
const store = new Vuex.Store({
    actions: {
        函数名 (store, 可选值) {
            // 异步代码, 把结果commit给mutations给state赋值
        }
    }
})
```

    - 具体代码:

```
const store  = new Vuex.Store({
    // ...省略state和mutations此处
    actions: {
        asyncAddCount(store, num){
            setTimeout(() => { // 1秒后, 异步提交给add的mutations
                store.commit('addCount', num)
            }, 1000)
        },
        asyncSubCount(store, num) {
            setTimeout(() => { // 1秒后, 异步提交给sub的mutations
                store.commit('subCount', num)
            }, 1000)
```

```
        }
    }
})
```

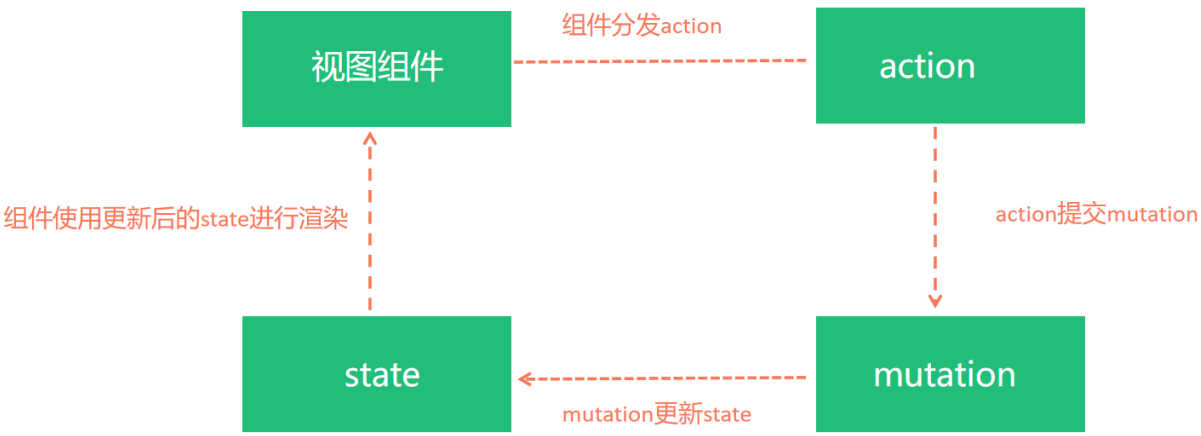
- 使用actions的两种方式
  - 直接使用
    - 语法:

```
this.$store.dispatch('actions函数名', 具体值)
```

  - 映射使用
    - 语法:

```
// 1. 拿到mapActions辅助函数
import { mapActions } from 'vuex'
export default {
    methods: {
        // 2. 把actions里方法映射到原地
        ...mapActions(['actions里的函数名'])
    }
}
```

- 小结
  - actions和mutations区别?
    - mutations里同步修改state
    - actions里放入异步操作
  - actions是否能操作state?
    - 不建议, 要commit给mutations(为调试工具可追踪)
  - actions和mutations里函数, 第一个形参分别是什么?
    - mutations的是state
    - actions的是store
  - actions使用方式?
    - this.$store.dispatch('actions方法名字', 值)
    - ...mapActions(['actions里的方法名']) 映射到原地使用
  - 视图组件, state, mutations, actions的关系是?



- **vuex-getters（计算属性）**
  - getters概念
    - vuex身上的全局状态-计算属性, 类似于computed
    - getters 依赖于 state中原始数据的变化，并返回计算后的新数据
  - 定义getters
    - 语法:

```
const store = new Vuex.Store({
    getters: {
        计算属性名 (state) {
            return 值给计算属性
        }
    }
})
```

    - 具体代码

```
const store = new Vuex.Store({
    // ...省略其他
    getters: {
        allCount(state) {
            return state.goodsList.reduce((sum, obj) => {
                if (obj.goods_state === true) { // 选中商品才累加数量
                    sum += obj.goods_count;
                }
                return sum;
            }, 0)
```

```
        },
        allPrice(state) {
            return state.goodsList.reduce((sum, obj) => {
                if (obj.goods_state) {
                    sum += obj.goods_count * obj.goods_price
                }
                return sum;
            }, 0)
        }
    }
})
```

- 使用getters的两种方式
  - 直接使用
    - 语法:

```
this.$store.getters.计算属性名
```

  - 映射使用
    - 语法:

```
// 1. 拿到mapGetters辅助函数
import { mapGetters } from 'vuex'
export default {
    computed: {
        // 2. 把getters里属性映射到原地
      ...mapGetters(['getters里的计算属性名'])
    }
}
```

- 小结
  - getters有什么用?
    - vuex里的计算属性, 属于全局计算属性, 类似computed
  - getters如何使用?
    - this.$store.getters.计算属性名
    - ...mapGetters(['getters里计算属性名'])

- **vuex-modules（分模块）**
  - 为何分模块

由于使用单一状态树，应用的所有状态会集中到一个比较大的对象。当应用变得非常复杂时，store 对象就有可能变得相当臃肿。

为了解决以上问题，Vuex 允许我们将 store 分割成模块（module）。每个模块拥有自己的 state、mutation、action、getter、甚至是嵌套子模块。

  - 代码上的对比

```
// 不分模块
const store = new Vuex.Store({
    state: {
        user: {
            name: "",
            age: 0,
            sex: ''
        },
        cart: {
            goodsList: []
        }
    },
    mutations: {},
    actions: {},
    getters: {},
    modules: {}
})
```

```
// 用户模块
const userModule = {
    state(){
        return {
            name: "",
            age: 0,
            sex: ''
        }
    },
    mutations: {},
    actions: {},
    getters: {}
}
```

```
// 购物车模块
const cartModule = {
    state(){
        return {
            goodsList: []
        }
    },
    mutations: {},
    actions: {},
    getters: {}
}
```

  - 创建modules模块对象
    - 对象里包含5个核心概念, 只有state改变为函数形式（目的是为了每个引用此模块的地方返回一个独立的新数据对象），其他核心无变化。
    - 语法:

```
// 用户模块对象
const userModule = {
    state(){
        return {
            name: "",
            age: 0,
            sex: ''
        }
    },
    mutations: {},
    actions: {},
    getters: {}
}
export default userModule
```

- 定义modules
  - 语法:

```
modules: {
    模块名: 模块对象
}
```

- 把2个模块对象, 引回到store里注册

```
import Vue from 'vue'
import Vuex from 'vuex'
import cartModule from './modules/cart'
import userModule from './modules/user'
Vue.use(Vuex)
const store = new Vuex.Store({
    modules: {
        user: userModule,
        cart: cartModule
    }
})
export default store
```

- state使用方式修改
  - 直接使用
    - 原语法:

```
this.$store.state.变量名
```

    - 分模块后语法:

```
this.$store.state.模块名.变量名
```

  - 映射使用
    - 原语法:

```
...mapState(['state里变量名'])
...mapState({'变量名': "state里变量名"})
```

    - 分模块后语法:

```
...mapState({
    '变量名': state => state.模块名.变量名
})
```

- 开启命名空间
  - 在模块对象内设置`namespaced: true`
    - 注意：是在模块对象中设置，不是在引入模块的对象中设置
  - 语法:

```
const moduleShopCar = {
    namespaced: true,
    state () {},
    mutations: {},
    actions: {},
    getters: {},
    modules: {}
```

```
}
```

- state使用方式修改
  - 直接使用
    - 原语法:

```
this.$store.state.变量名
```

    - 分模块后语法:

```
this.$store.state.模块名.变量名
```

    - 分模块并开启命名空间后语法（与不开启命名空间比无变化）:

```
this.$store.state.模块名.变量名
```

  - 映射使用
    - 原语法:

```
...mapState(['state里变量名'])
...mapState({'变量名': "state里变量名"})
```

    - 分模块后语法:

```
...mapState({
    '变量名': state => state.模块名.变量名
})
```

    - 分模块并开启命名空间后语法:

```
...mapState("模块名", ['state变量名'])
```

- mutations使用方式修改
  - 直接使用
    - 原语法:

```
this.$store.commit("mutations里的函数名", 具体值)
```

    - 分模块并开启命名空间后语法:

```
this.$store.commit("模块名/mutations里的函数名", 具体值)
```

  - 映射使用
    - 原语法:

```
...mapMutations(['mutations里方法名'])
```

    - 分模块并开启命名空间后语法:

```
...mapMutations("模块名", ['mutations里方法名'])
```

- actions使用方式修改
  - 直接使用
    - 原语法:

```
this.$store.dispatch("actions里的函数名", 具体值)
```

    - 分模块并开启命名空间后语法:

```
this.$store.dispatch("模块名/actions里的函数名", 具体值)
```

  - 映射使用
    - 原语法:

```
...mapActions(['actions里方法名'])
```

- 分模块并开启命名空间后语法:

```
...mapActions("模块名", ['actions里方法名'])
```

- getters使用方式修改
  - 直接使用
    - 原语法:

```
this.$store.getters.计算属性名
```

- 分模块并开启命名空间后语法:

```
this.$store.getters['模块名/计算属性名']
```

  - 映射使用
    - 原语法:

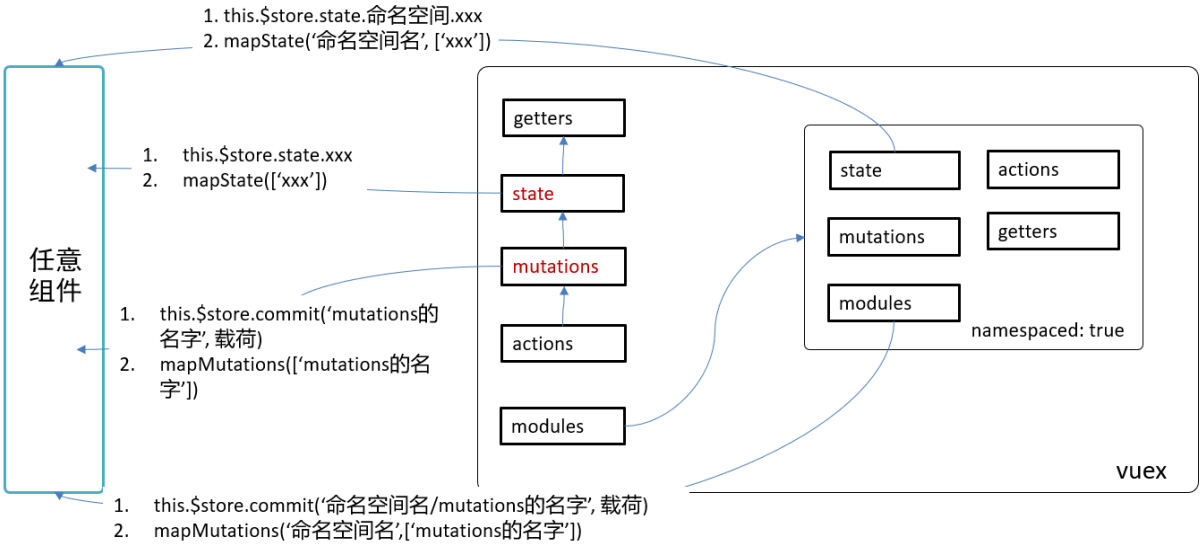
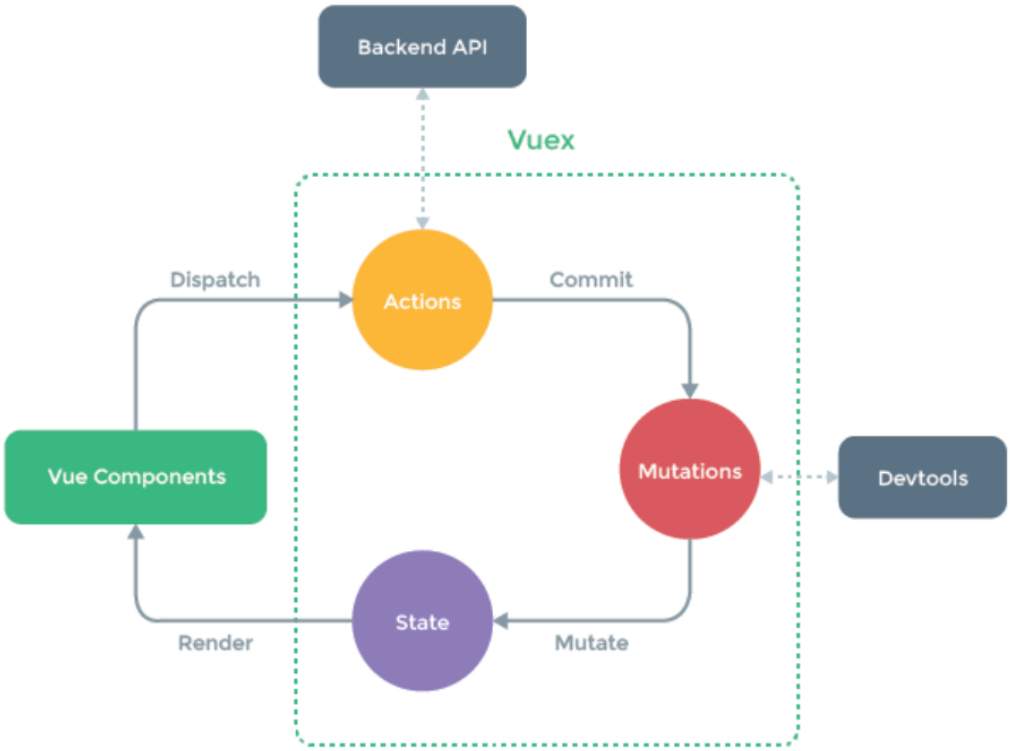
```
...mapGetters(['getters里计算属性名'])
```

- 分模块并开启命名空间后语法:

```
...mapGetters("模块名", ['getters里计算属性名'])
```

- 小结
  - 为什么分模块?
    - 集中式管理项目过大, 变量过多, 会导致state臃肿, 难以维护
  - 如何分模块?
    - 定义模块对象, state变成函数返回对象形式, 每个模块都有state/mutations/actions/getters/modules
  - 根store如何注册?

    - modules里 { 模块名: 模块对象 }
- 分模块不开启命名空间对什么有影响?
    - 对state的取值方式有影响, 对其他暂无影响
- 分模块不开启命名空间state如何取值?
    - 在组件使用的时候, 要state.模块名.变量名
- state和mutations, 在根store和开启命名空间里的区别?



- 整个vuex的体系是?